

3주차 병렬화 가능성 분석과 벤치마킹

PART 1

병렬화 가능성 분석

병렬 구조·직렬 구간·전송 고정비를 분석하고

정확성 및 측정 범위를 통제된 CPU/GPU Baseline을 설계

- ❖ **전송 고정비** : 데이터를 보낼 때 데이터의 크기(용량)와 상관없이 '무조건 기본적으로 들어가는 시간이나 비용'
- ❖ **Baseline** : 시간 숫자 하나가 아니라 입력, 정확성, 측정 범위와 반복 규칙을 고정한 비교 설계이다.

Host와 Device

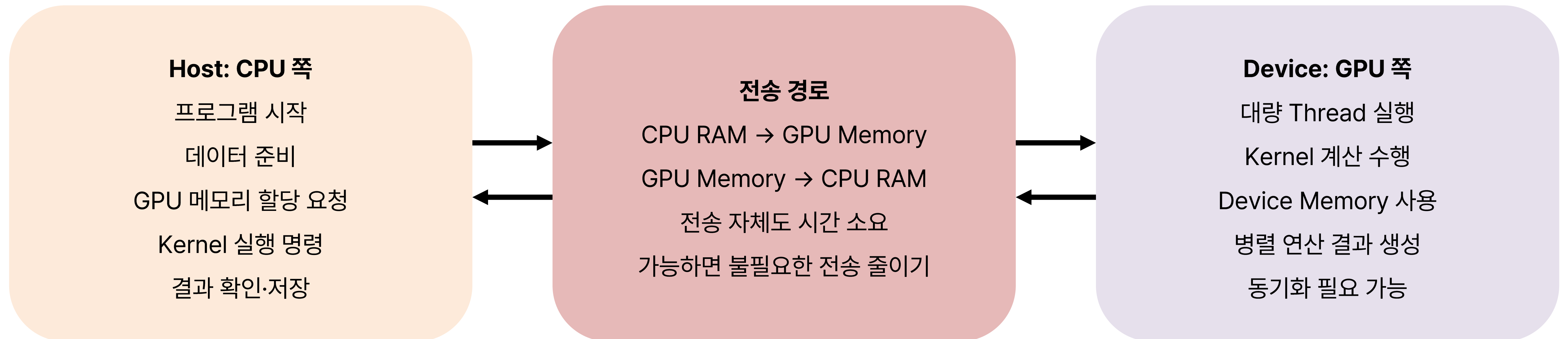


사진 한 장은 CPU가 빠르는데 1,000장은 왜 달라질까?

"손님 1명에겐 셰프 1명이 빠르지만, 1,000명의 단체 손님에겐 100명의 조리 보조가 유리합니다."

CPU (Central Processing Unit)

- 비유: 천재 셰프 1명
- 특징: 복잡한 고급 코스 요리를 순서대로 빠르게 처리
- 구조: 강력한 성능을 가진 소수의 코어 (Sequential)
- 최적화: 소량의 복잡한 데이터 및 연속적인 작업

GPU (Graphics Processing Unit)

- 비유: 조리 보조 100명
- 특징: 대량의 마늘 까기, 채소 다듬기를 동시에 해결
- 구조: 단순 작업에 특화된 수천 개의 코어 (Parallel)
- 최적화: 대규모의 단순 반복 데이터 및 병렬 작업

❖ 작업 규모에 따른 효율성 분석 : '준비 시간(고정비)'의 법칙

소형 작업 : 조리 보조 100명을 모으고 역할을 나누는 초기 준비 시간(고정비)이 더 큼. 셰프 1명이 즉시 조리하는 것이 효율적입니다.

대형 작업 (AI·빅데이터) : 대규모 작업에서는 초기 고정비(준비 시간)가 발생하더라도,
동시 다발적 병렬 처리를 통해 얻는 시간 절감 효과가 압도적으로 유리합니다.

오늘의 수업은 세 가지 질문에 답한다

정의 → 계산 → 측정 → 프로젝트

무엇을 GPU로 옮길까?

- 데이터·작업 병렬성과 직렬 구간 분석
- 계산·메모리·제어 집약도 판단
- Amdahl·Gustafson·Strong/Weak Scaling
- 고정비와 Crossover

무엇을 어떻게 썰까?

- Host timer와 CUDA Event의 시간측 구분
- 비동기 실행·동기화 범위 통제
- Cold/Warm·반복 측정·중앙값과 꼬리값
- Kernel-only·E2E·공정한 CPU Baseline

무엇을 제출할까?

- 입력·출력·병렬 단위·전송 경계 정의
- 예상 병목과 정확성 합격 조건 명시
- Latency 또는 Throughput 주지표 선택
- 재현 가능한 설계서와 Baseline 작성

산출물 → 프로젝트 설계서 + 재현 가능한 CPU/GPU Baseline

오늘의 수업은 세 가지 질문에 답한다

정의 → 계산 → 측정 → 프로젝트

무엇을 GPU로 옮길까?

- 데이터·작업 병렬성과 직렬 구간 분석
- 계산·메모리·제어 집약도 판단
- Amdahl·Gustafson
- 고정비와 Crossover

암달의 법칙(Amdahl)

작업 크기가 고정되어 있을 때, 아무리 일꾼(코어)을 늘려도 혼자 해야 하는 일 때문에 성능 향상에 한계를 의미

구스타프슨의 법칙(Gustafson)

작업 크기가 늘어나고 코어가 늘어나면 성능이 향상되는 것을 의미

• Kernel-only·EZE·종장한 CPU Baseline

• 재현 가능한 설계서와 Baseline 작성

산출물 → 프로젝트 설계서 + 재현 가능한 CPU/GPU Baseline

오늘의 수업은 세 가지 질문에 답한다

정의 → 계산 → 측정 → 프로젝트

무엇을 GPU로 옮길까?

- 데이터·작업 병렬성과 직렬 구간 분석
- 계산·메모리·제어 집약도 판단
- Amdahl·Gustafson
- 고정비와 Crossover

무엇을 어떻게 썰까?

- Host timer와 CUDA Event의 시간측 구분
- 비동기 실행·동기화 범위 통제
- Cold/Warm·반복 측정·중앙값과 꼬리값
- Kernel-only·E2E·공정한 CPU Baseline

무엇을 제출할까?

- 입력·출력·병렬 단위·전송 경계 정의
- 예상 병목과 정확성 합격 조건 명시
- Latency 또는 Throughput 주지표 선택
- 재현 가능한 설계서와 Baseline 작성

Cold 실행

처음 실행하는 상태 (메모리에 데이터 없음)

Warm 실행

이미 실행한 적이 있는 상태 (메모리에 데이터 있음)

산출물

Baseline

오늘의 수업은 세 가지 질문에 답한다

정의 → 계산 → 측정 → 프로젝트

무엇을 GPU로 옮길까?

- 데이터·작업 병렬성과 직렬 구간 분석
- 계산·메모리·제어 집약도 판단
- Amdahl·Gustafson
- 고정비와 Crossover

Latency

입력과 출력 사이에 대한 시간 간격

Throughput

시간 단위 당 데이터 처리량

- Kernel-only·EZE·종정인 CPU Baseline

무엇을 제출할까?

- 입력·출력·병렬 단위·전송 경계 정의
- 예상 병목과 정확성 합격 조건 명시
- Latency 또는 Throughput 주 지표 선택
- 재현 가능한 설계서와 Baseline 작성

산출물 → 프로젝트 설계서 + 재현 가능한 CPU/GPU Baseline

병렬성은 Thread 수가 아니라 독립적으로 진행 가능한 일의 구조다

1. 정의·개념 : 일이 어떻게 쪼개져 있는가? <병렬성 구조에 대한 파악>

- ❖ 데이터 병렬성 (같은 일, 다른 재료): 동일한 행동을 서로 다른 데이터에 적용
- ❖ 작업 병렬성 (다른 일, 동시에 진행): 서로 다른 종류의 일을 동시에 처리
- ❖ 직렬 구간 (기다려야 하는 필수 코스): 앞 작업이 끝나야만 시작할 수 있는 ' 병렬화 불가능한 구간'

2. 작동 원리·형식화 : 어떻게 동시에 돌릴 것인가? <문제 점검에 대한 확인>

- ❖ 의존성 그래프 (관계도 그리기) : 연산을 노드, 의존성을 화살표로 그리면 동시에 실행 가능한 노드가 보인다.
- ❖ 쓰기 충돌과 재설계 (충돌 방지) : 읽기 공유는 가능하지만 같은 출력에 쓰는 순간 Atomic·단계 분리·재설계가 필요하다.
- ❖ 배보다 배꼽이 큰 고정비 (오버헤드) : 병렬 단위(일의 크기)가 너무 작으면 스케줄링과 Kernel Launch 고정비가 계산보다 커진다.

3. 가정·한계·판단 : 진짜 빨라질 수 있는가? <성능에 대한 검증>

- ❖ 독립성은 읽기/쓰기 충돌과 실행 순서 의존성이 모두 없다는 뜻이다.
- ❖ 병렬성이 많아도 메모리 공급과 동기화가 실제 가속을 제한할 수 있다.
- ❖ 설계 판단: 입력·출력 주소표와 의존성 그래프를 만든 뒤 Thread·Block·Batch가 맡을 단위를 정한다.

독립성은 주소와 의존성으로 증명한다

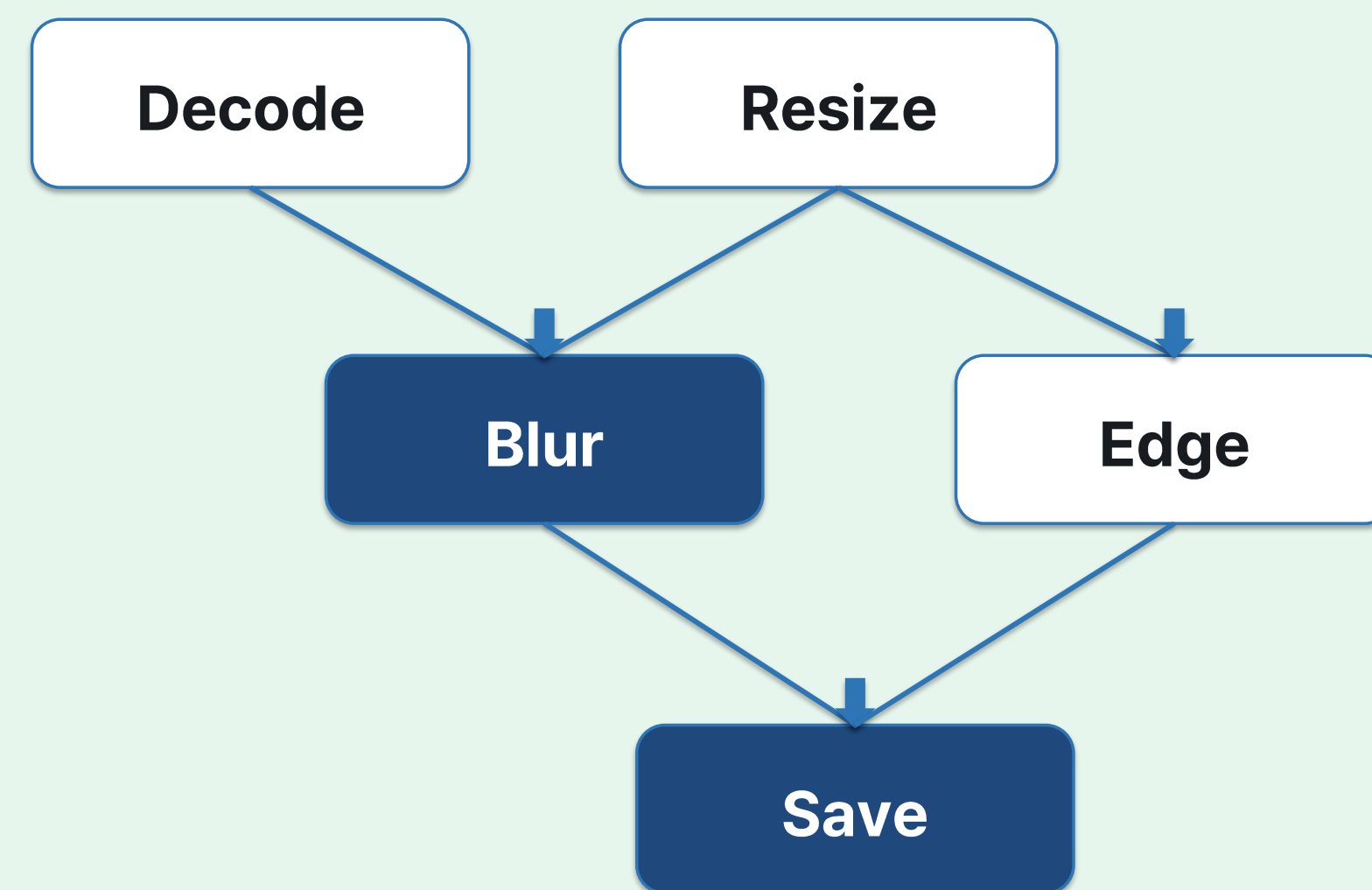
반복문의 모양이 아니라 각 작업이 읽고 쓰는 주소와 선행 결과를 확인한다.

① 입력·출력 주소표

작업 단위	읽는 주소	쓰는 주소	충돌 판단
밝기 $y[i]$	$x[i]$	$y[i]$	독립
3x3 Blur $y[i]$	$x[i-1:i+1]$	$y[i]$	읽기 공유
누적합 $y[i]$	$y[i-1], x[i]$	$y[i]$	단계 의존

$i \neq j$ 일 때 동일한 출력 주소를 쓰지 않으면 원소별 독립성을 확보할 수 있다.

② 의존성 그래프



화살표가 없는 노드는 같은 시간 구간에 실행할 수 있다.

읽기 공유는 가능하지만 같은 출력에 쓰는 순간 Atomic·단계 분리·재설계가 필요하다.

(실습 W03-L01) 환경과 CUDA 인덱스 첫 실행

```
w3_source = r"""
extern "C" __global__ void affine(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = 2.0f * x[i] + 1.0f;
}
"""

w3_kernel = make_kernel(w3_source, "affine")
```

thread 0~7은 각각 어떤 출력을 만들까?

예상:

관측 후 수정:

(실습 W03-L01) 환경과 CUDA 인덱스 첫 실행

```
w3_source = r"""
extern "C" __global__ void affine(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) y[i] = 2.0f * x[i] + 1.0f;
}
"""

w3_kernel = make_kernel(w3_source, "affine")
```

thread 0~7은 각각 어떤 출력을 만들까?

입력: [0. 1. 2. 3. 4.]

GPU 전체 출력 (32개): [1. 3. 5. 7. 9. 0.]

(실습 W03-L01) 환경과 CUDA 인덱스 첫 실행

```
w3_source = r"""
extern "C" __global__ void affine(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    y[i] = 2.0f * x[i] + 1.0f;
}
"""

w3_kernel = make_kernel(w3_source, "affine")
```

If($i < n$) 조건을 제거하면 어떤 입력에서 위험할까?

안전장치가 없다면?

데이터가 배정되지 않은 남은 일꾼들(예: 100번~255번 일꾼)도 자기 차례인 줄 알고 메모리에 접근하게 된다.

존재하지 않는 배열 공간인 $x[100]$ 의 값을 읽으려고 하거나, 엄한 메모리 주소인 $y[100]$ 자리에 계산 결과를 강제로 써버린다.

이로 인해 프로그램이 갑자기 강제 종료되는 메모리 접근 오류(Out-of-bounds/Segmentation Fault)가 발생하거나,

컴퓨터의 다른 중요한 메모리 데이터를 오염시키는 치명적인 버그가 발생할 수 있다.

(실습 W03-L01) 환경과 CUDA 인덱스 첫 실행

```
w3_source = r"""
extern "C" __global__ void affine(const float* x, float* y, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    y[i] = 2.0f * x[i] + 1.0f;
}
"""

w3_kernel = make_kernel(w3_source, "affine")
```

If($i < n$) 조건을 제거하면 어떤 입력에서 위험할까?

입력: [0. 1. 2. 3. 4.]

GPU 전체 출력 (32개): [1. 3. 5. 7. 9. 1.]

유효 데이터 크기(5개)를 넘어선 5번부터 31번 인덱스의 모든 공간이 원래 나오면 안 되는 값인 1.로 오염된 것을 확인할 수 있다.

병렬성은 Thread 수가 아니라 독립적으로 진행 가능한 일의 구조다

◆ 수식으로 보는 병렬 컴퓨팅의 진실

$$[\text{총 시간} = \text{직렬 경로} + \text{병렬 경로} + \text{통신·동기화 비용}]$$

직렬 경로

⇒ 아무리 보조 작업자가 많아도 절대 줄일 수 없는 최소한의 필수 시간.

병렬 경로

⇒ 일손(Core/Thread)을 늘릴수록 시간이 획기적으로 줄어드는 구간.

통신·동기화 비용

⇒ 일손이 늘어날수록 서로 눈치 보며 기다리고 확인하는 소통 비용(오버헤드)은 오히려 늘어남.

일손을 1,000명으로 늘려 '병렬 경로' 시간을 0에 가깝게 줄이더라도,
'직렬 경로'와 '통신·동기화 비용'이라는 한계치 때문에 전체 속도가 무한정 빨라질 수는 없다.

GPU 가속은 연산량의 총량이 아니라, [연산·이동·제어]의 비율로 결정된다.

◆ 3가지 핵심 집약도 분석

정성적 이식 우선순위 평가 (Qualitative Model)

- 정확한 수치 모델(Roofline Model) 분석 전, 이식 후보를 선별하는 출발점으로 활용
- [평가표 지표]: 연산/Byte(AI), 데이터 재사용성, 분기 규칙성, 입력 데이터 크기 → 종합 점수화

1. 계산 집약 (Compute-Bound) [GPU 최적]

- 데이터 1 Byte당 수행하는 연산이 압도적으로 많은 구조
- 비유 : 재료 하나를 가져와서 1시간 동안 화려하게 조리하는 요리
- 체크 : 데이터 재사용성이 높을수록 GPU에 매우 유리

2. 메모리 집약 (Memory-Bound) [주의 필요]

- 연산 능력보다 데이터 공급 속도(대역폭)가 발목을 잡는 구조
- 비유 : 조리는 1초 만에 끝나는데, 재료 배달오는 데 10분 걸리는 요리
- 체크 : 데이터 입력 크기가 너무 작으면 이동 비용 때문에 GPU가 손해

3. 제어 집약 (Control-Bound) [GPU 기피]

- 조건문(if-else)이 많아 데이터마다 처리 경로가 불규칙하게 갈리는 구조
- 비유 : 손님마다 요구사항이 다 달라 조리 보조들이 제각각 따로 노는 상황
- 체크 : 불규칙한 분기와 간접 주소는 계산량이 많아도 GPU 효율을 급감시킴

이식 후보는 네 지표를 같은 표에서 점수화한다

정확한 Roofline 분석 전에는 연산/Byte·재사용·분기 규칙성·입력 크기로 후보를 선별한다.

후보 연산	독립성	연산/Byte	재사용성	분기 규칙성	입력 규모	우선순위
밝기 조정	높음	낮음	낮음	높음	작음~중간	2
3×3 Blur	높음	중간	높음	높음	중간~큼	1
Prefix Sum	단계형	중간	중간	높음	큼	3

연산/Byte

- 이동한 Byte당 유용한 연산량

재사용성

- 가져온 데이터를 몇 번 활용하는가

분기 규칙성

- Warp의 활성 Lane이 유지되는가

입력 크기

- 고정비를 상쇄할 충분한 작업량인가

점수는 "정답"이 아니라 이식 실험의 순서를 정하는 정성적 필터다.

10분 중 2분이 직렬이면 최대 가속은 5배다

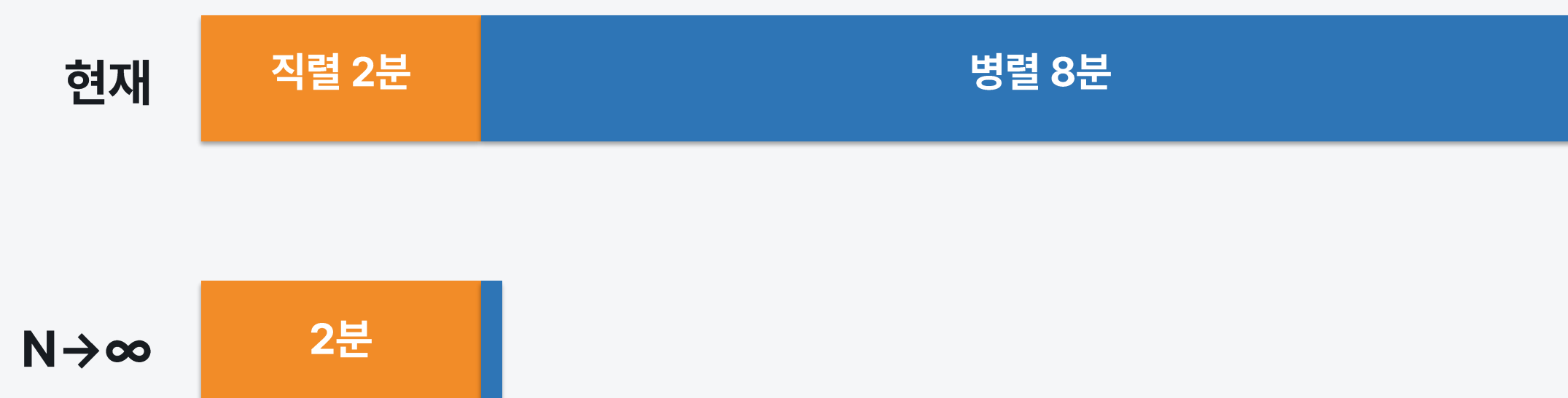
Amdahl의 법칙은 문제 크기를 고정하고 직렬 비율이 만드는 가속 상한을 본다.

Amdahl's Law

$$S(N) = 1 / ((1-p) + p/N)$$

- p: 측정 범위에서 병렬화 가능한 시간 비율
- N: 병렬부의 실제 가속 배수 또는 처리 자원
- **$N \rightarrow \infty$ 이어도 $(1-p)$ 는 남는다**

10분 작업의 시간 구성



$$\text{최대 가속} = 10\text{분} / 2\text{분} = 5\text{배}$$

p는 코드 줄 수가 아니라 동일한 측정 범위의 실제 시간 비율이다.

병렬부를 극단적으로 최적화할수록 남은 직렬부가 다음 병목이 된다.

다음 한 시간은 “가장 느린 코드”가 아니라 전체시간을 가장 줄이는 곳에 쓴다

변경 전후의 전체 경로를 다시 합산해야 최적화 투자 우선순위를 판단할 수 있다.

기준 실행

직렬 2분 + Kernel 8분

총 10분

비교 기준을 고정

선택 A · Kernel 개선

8분 → 4분

총 6분

전체시간 4분 감소

선택 B · 직렬부 개선

2분 → 1분

총 9분

전체시간 1분 감소

현재 조건에서는 Kernel 개선이 우선이다. 단, Kernel이 충분히 빨라지면 직렬부가 다음 우선순위가 된다.

같은 10분에 100장 대신 800장을 처리하는 것도 가속

Gustafson의 법칙은 실행시간을 고정하고 자원과 함께 문제 규모를 키울 때의 확장성을 본다.

Gustafson's Law

$$S_G(P) = P - \alpha(P-1)$$

- P: 병렬 자원 또는 처리 규모
- α : 고정 시간 안에서 직렬 구간의 비율
- 질문: 같은 시간 동안 얼마나 더 큰 문제를 처리하는가?

600초 동안 완료한 이미지 수

CPU

100장

0.1667 images/s

GPU

800장

1.333 images/s

처리량 기준 8배

Latency 서비스는 고정 문제 관점, 대규모 배치 처리는 문제 규모 확장 관점을 우선한다.

총 1,024장과 GPU당 1,024장은 다른 실험이다

Strong scaling은 총 문제량을 고정하고, Weak scaling은 장치당 문제량을 고정한다.

GPU 수가 늘 때 무엇을 고정하는가?

실험	1 GPU	2 GPU	4 GPU
Strong 총량 고정	총 1,024	총 1,024	총 1,024
Weak 장치당 고정	GPU당 1,024 총 1,024	GPU당 1,024 총 2,048	GPU당 1,024 총 4,096

Strong Scaling

- 같은 총량의 완료시간이 얼마나 줄어드는가
- 통신·동기화가 커지면 효율이 떨어진다

Weak Scaling

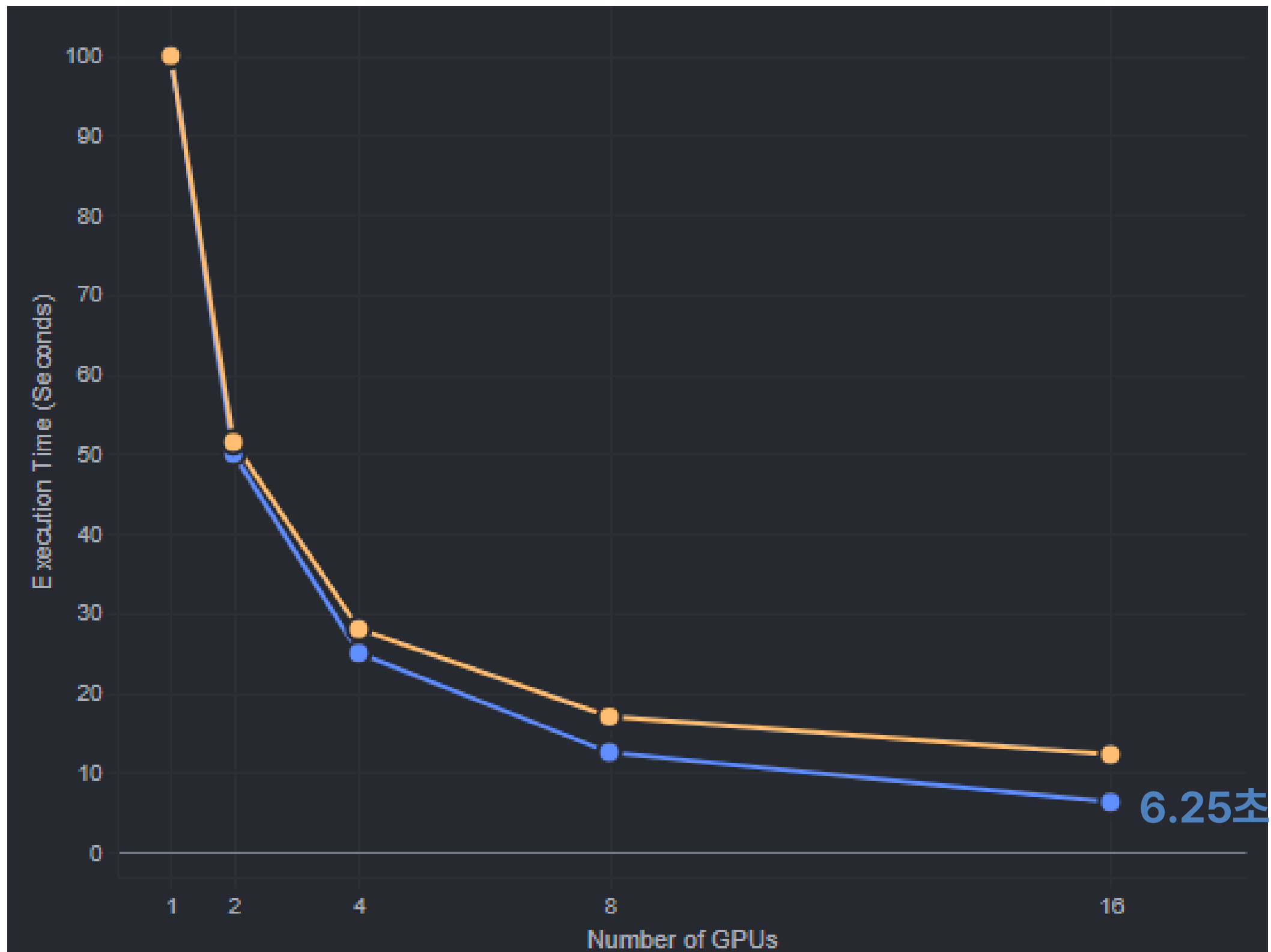
- 장치당 일을 유지할 때 전체량을 얼마나 키우는가
- 이상적이면 완료시간은 거의 일정하다

그래프를 보기 전에 x축, y축, 고정한 조건을 먼저 적어야 해석이 가능하다.

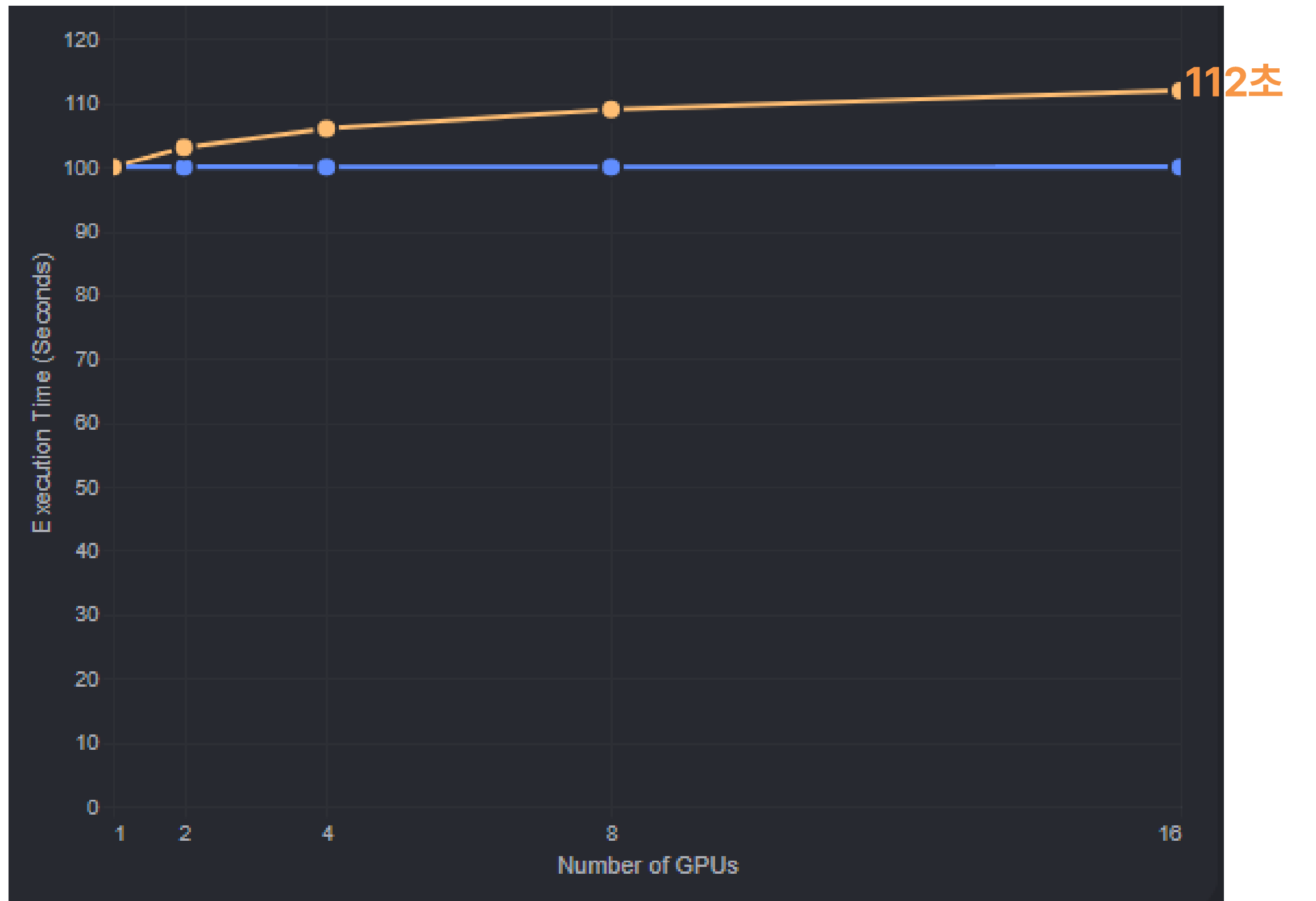
총 1,024장과 GPU당 1,024장은 다른 실험이다

Strong scaling은 총 문제량을 고정하고, Weak scaling은 장치당 문제량을 고정한다.

Strong Scaling 그래프 예시



Weak Scaling 그래프 예시



GPU는 계산 전에 고정비를 먼저 지불한다

PCIe 전송·Kernel Launch·동기화 비용은 입력이 작아도 먼저 발생한다.

End-to-End 시간 모델

$$T_{\text{GPU}}(N) = T_{\text{fixed}} + T_{\text{H2D}} + T_{\text{K}}(N) + T_{\text{D2H}}$$

- T_{fixed} : Context·Launch·동기화 등 고정비
- H2D/D2H: 데이터 이동량과 대역폭의 함수
- $T_{\text{K}}(N)$: 입력이 커질수록 증가하는 Kernel 계산
- 데이터 상주·Batching은 고정비의 비중을 낮춘다

축소 수치 모델

$$\text{CPU} = 0.02N \text{ ms}$$

$$\text{GPU} = 1.5\text{ms} + 0.002N \text{ ms}$$

GPU

고정비 1.5ms

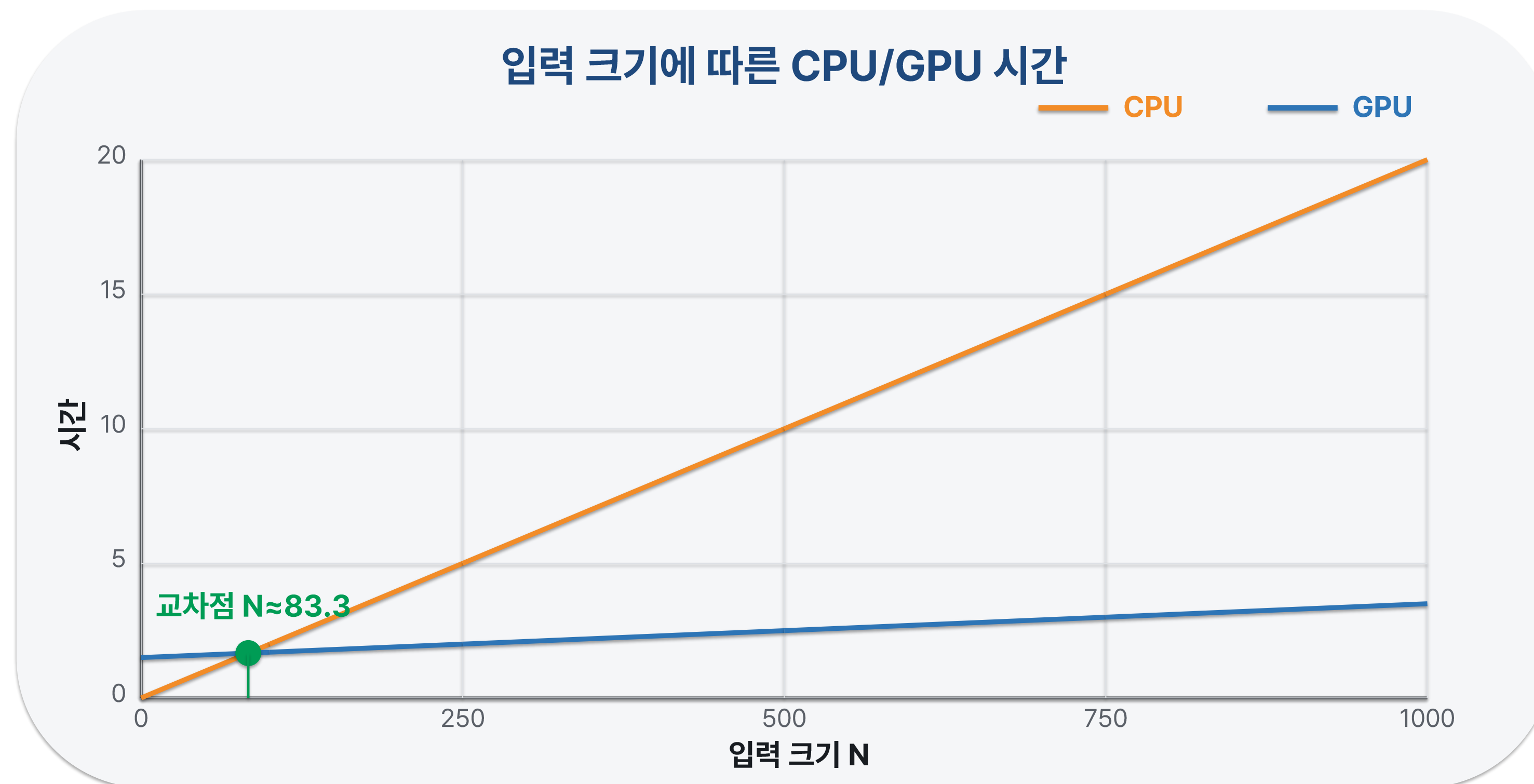
0.002N

작은 N에서는 1.5ms가 계산 이득을 삼킨다.

GPU가 느린 것이 아니라 “고정비를 상쇄할 만큼 일이 크지 않은” 경우가 있다.

N≈84부터 GPU가 이기는 축소 모델

한 크기만 재지 말고 입력 크기 Sweep으로 두 곡선이 만나는 지점을 찾는다.



손으로 교차점 계산

$$0.02N = 1.5 + 0.002N$$

$$0.018N = 1.5$$

$$N \approx 83.3$$

- N=8: CPU 0.16ms < GPU 1.516ms
- 정수 입력에서 첫 GPU 우세 후보: N=84

교차점은 GPU·전송 방식·Batch·데이터 상주 여부가 바뀌면 다시 측정해야 한다.

GPU 이식 판단은 세 문장으로 요약할 수 있어야 한다

정의한 것

- 독립 작업 단위와 출력 주소 소유권
- 직렬 구간과 전송 경계
- Compute/Memory/Control 후보 병목

계산한 것

- Amdahl 최대 가속 상한
- Gustafson 처리량 확장 관점
- 고정비 모델과 Crossover 입력 크기

결정한 것

- GPU로 먼저 옮길 후보
- CPU에 남길 작은 입력 구간
- 다음 단계에서 측정할 범위와 지표

"GPU가 빠를 것 같다"가 아니라 조건·근거·반례가 포함된 판단 문장을 남긴다.

PART 2

비교 가능한 시간을 재는 법

비동기 실행의 시간측을 구분하고,

Cold/Warm·반복 통계·정확성·CPU 기준선을 하나의 실험 조건으로 고정.

속도 숫자에는 실행 환경이 포함된다

GPU 이름과 CUDA·Driver·Compiler·dtype·반복 조건이 없으면 다른 사람이 같은 실험을 재현할 수 없다.

Colab 에서 확인

```
!nvidia-smi  
!nvcc --version  
!python --version
```

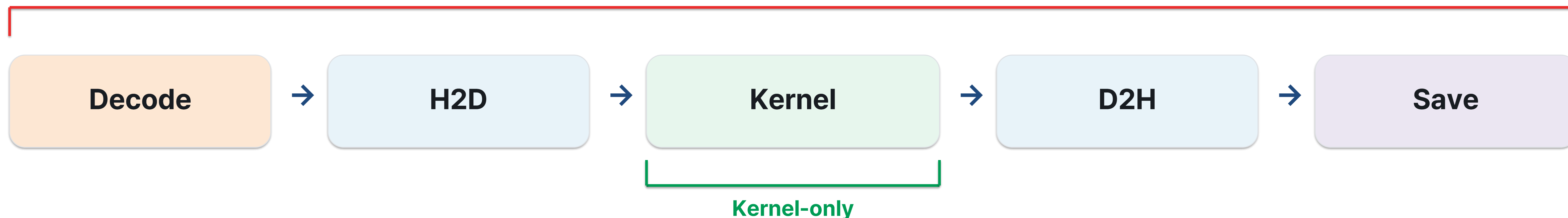
Baseline 메타데이터

GPU / Driver	실행 시 GPU / Driver 기록
CUDA / Compiler	nvcc / Python 버전 기록
입력 설정	shape, dtype=FP32, seed=42
측정 설정	Cold 1회 + Warm-up 5 + 반복 200
출력 설정	median, p95, correct, max_error

환경과 측정 조건이 바뀌면 성능 차이를 코드 변경의 효과라고 단정할 수 없다.

Decode→H2D→Kernel→D2H→Save 중 어디까지 썰 것인가?

End-to-End · 사용자가 기다리는 전체 구간



Kernel-only

- GPU Kernel 또는 지정 GPU 구간만 측정
- 커널 최적화 효과를 분리

Stream/Stage

- 특정 Stream의 복사+Kernel 단계
- Pipeline 병목을 분리

End-to-End

- 입력 준비부터 사용자 결과까지
- 제품·서비스 성능을 판단

시간 숫자에는 반드시 “시작점·종료점·포함 단계”를 한 문장으로 붙인다.

Host timer와 CUDA Event는 서로 다른 시계를 본다

Host Timer · CPU Wall Clock

- CPU 코드 구간의 경과시간을 측정
- 비동기 Launch만 감싸면 제출시간만 보일 수 있음
- E2E 측정에는 완료 조건을 명시적으로 포함
- 여러 Stream 전체 Wall time에 적합할 수 있음

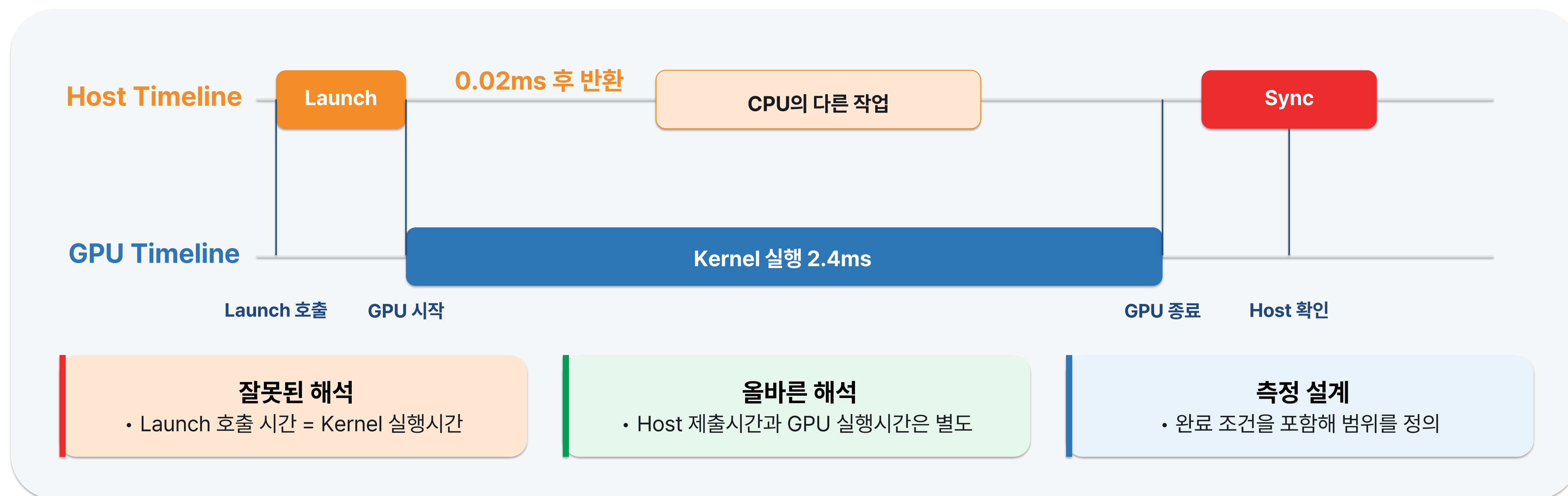
CUDA Event · GPU Stream Clock

- 특정 Stream에서 Event가 도달한 시점을 기록
- start/stop Event 사이 GPU 구간을 측정
- stop 완료를 기다린 뒤 elapsed 계산
- 다른 Stream의 모든 작업을 자동 포함하지 않음

측정 도구보다 먼저 “어떤 시간측과 어떤 완료 조건을 볼 것인가”를 정의한다.

Host의 0.02ms는 실행시간이 아니라 제출시간일 수 있다

Launch 반환 시점, Kernel 종료 시점, 동기화 시점을 분리해서 본다.



비동기 실행에서는 "호출이 반환됐다"와 "작업이 끝났다"가 같은 시점이 아니다.

같은 Stream의 시작·끝 Event 사이를 측정하고 완료를 기다린다

Kernel-only 측정 패턴

```
cudaEventRecord(start, stream);

// GPU에서 대량의 데이터를 병렬로 연산하고 처리하는 코드
kernel<<<grid, block, 0, stream>>>(...);

cudaEventRecord(stop, stream);

cudaEventSynchronize(stop);
cudaEventElapsedTime(&ms, start, stop);
```

측정이 성립하는 조건

- 1 start와 stop을 같은 Stream에 기록
- 2 stop Event가 완료될 때까지 기다림
- 3 두 Event 사이에 포함된 작업만 해석
- 4 Multi-Stream 전체시간은 별도 완료 신호 설계

cudaDeviceSynchronize는 정확할 수 있지만 불필요한 전역 대기로 원래 겹침을 왜곡할 수 있다.

첫 실행 8.2ms와 이후 2.0ms는 서로 다른 질문에 답한다

Cold 실행 · 첫 사용자 비용

8.2 ms

- CUDA Context 생성
- JIT/모듈 로딩·메모리 초기화
- 캐시 채움·GPU 클럭 전환
- 서버리스·단발 작업에서는 실제 비용

Warm 실행 · 정상 상태 비용

2.0 ms

- 준비가 끝난 반복 작업의 분포
- Warm-up 횟수와 반복 횟수를 기록
- Buffer 재사용·입력 생성 조건 고정
- Cold와 같은 열에 섞지 않고 별도 보고

첫 실행을 “무조건 버리는 것”이 아니라 사용자 시나리오에 따라 Cold와 Warm을 각각 평가한다.

[2.1, 2.0, 2.0, 2.2, 2.0]에서 대표값과 꼬리를 고른다

측정 샘플

2.1 · 2.0 · 2.0 · 2.2 · 2.0 ms

정렬

2.0 · 2.0 · 2.0 · 2.1 · 2.2

median = 2.0 ms

Median

- 대표적인 정상 실행
- 이상치에 평균보다 강함

p90 / p95

- 느린 꼬리 지연
- 사용자 경험·SLA에 중요

NumPy linear p90 = 2.16 ms

- 백분위수 보간 규칙을 함께 기록
- 평균·표준편차만으로 꼬리를 숨기지 않기
- 다른 프로세스·전원·온도 조건도 메타데이터화

한 번의 최고 기록이 아니라 반복 가능한 분포가 성능 증거다.

[실습 W03-L02] 세 가지 타이머와 첫 실행

```
timer_n = 1 << 20
timer_repeat = 30
if GPU_OK:
    timer_x = cp.ones(timer_n, dtype=cp.float32)
    timer_y = cp.empty_like(timer_x)
    timer_args = (timer_x, timer_y, np.int32(timer_n))
    w3_kernel(((timer_n+255)//256,), (256,), timer_args)
    cp.cuda.runtime.deviceSynchronize()

if GPU_OK:
    start_event, stop_event = cp.cuda.Event(), cp.cuda.Event()
    start_event.record()
    w3_kernel(((timer_n+255)//256,), (256,), timer_args)
    stop_event.record()
    stop_event.synchronize()
    elapsed_ms = cp.cuda.get_elapsed_time(start_event, stop_event)
    print("직접 켜 GPU Event 구간(ms):", elapsed_ms)

run_w3_timers()
```

호출시간, 완료시간, GPU Event가 같을까요?

예상:

관측 후 수정:

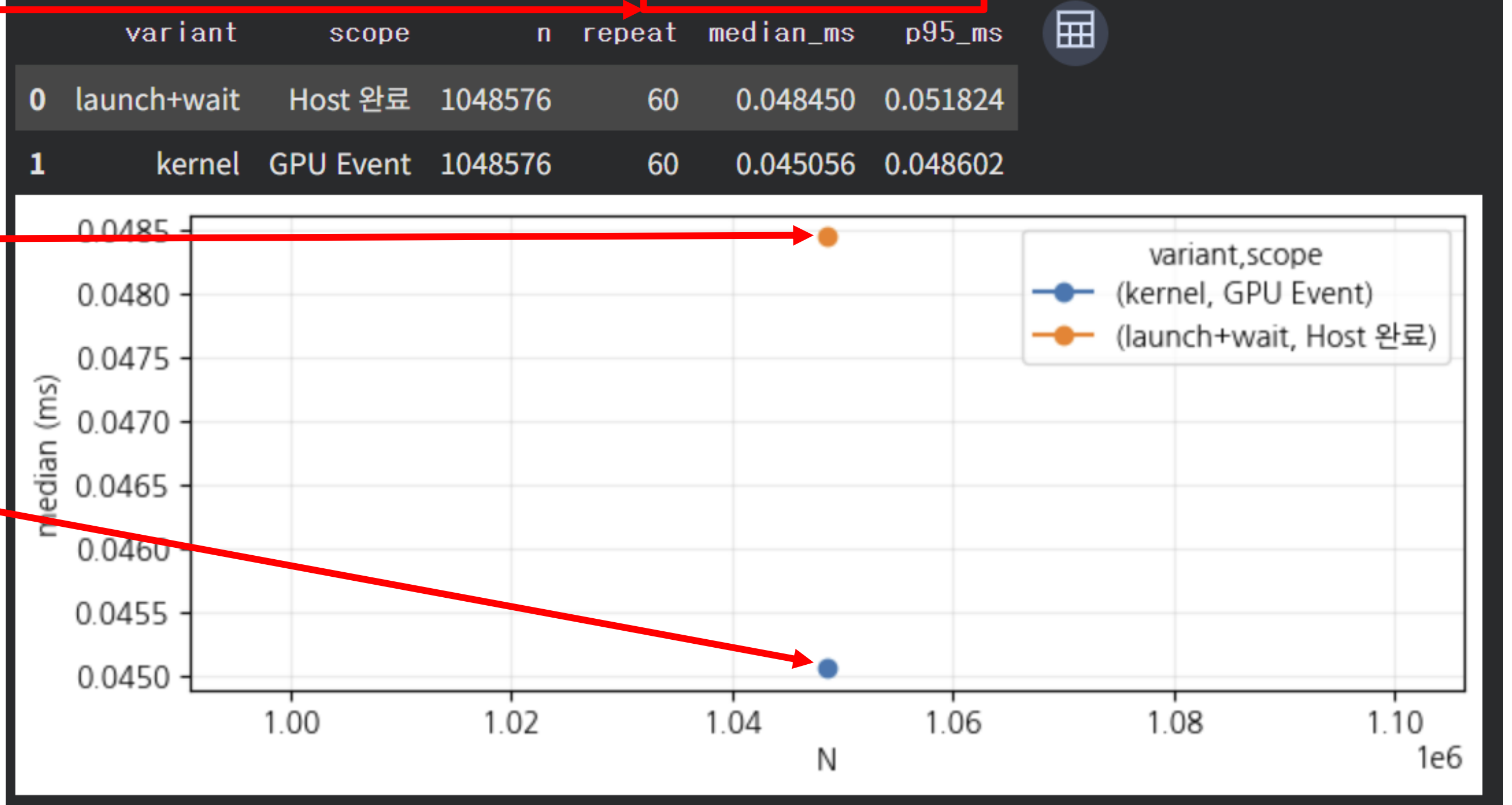
[실습 W03-L02] 세 가지 타이머와 첫 실행

호출시간 : 0.006820 ms

완료시간 : 0.048450 ms

GPU Event : 0.045056 ms

직접 켜 GPU Event 구간(ms): 0.09804800152778625
이 커널의 첫 호출+완료(ms): 0.0797180000517983 (이미 컴파일했다면 JIT 비용 없음)
완료를 기다리지 않은 호출시간 median(ms): 0.0068199999532225775 — GPU 실행시간 아님



호출시간, 완료시간, GPU Event가 같을까요?
서로 다르다.

GPU 연산 메커니즘의 특성상 세 시간은 일치하지 않으며
호출시간 < GPU Event < 완료시간 순으로 차이가 난다.

호출/Event : 호출시간은 명령만 내리고 넘어가는 비동기 방식이라 가장 짧고, GPU Event는 GPU가 물리적으로 순수 연산한 시간이다.
완료시간 : 완료시간은 CPU와 GPU 간의 통신 및 대기 오버헤드가 추가되므로 순수 GPU Event 시간보다 항상 더 오래 걸린다.

[실습 W03-L02] 세 가지 타이머와 첫 실행

직접 켜 GPU Event 구간(ms): 0.09804800152778625

1. 가장 작은 숫자가 커널 시간인가?

2. 첫 호출에 무엇이 포함되었나?

호출
서로
GPU
호출
호출
완료

[실습 W03-L02] 세 가지 타이머와 첫 실행

직접 잰 GPU Event 구간(ms): 0.09804800152778625

1. 가장 작은 숫자가 커널 시간인가?

아니다. 가장 작은 값(0.006820ms)은 CPU가 명령만 던지고 넘어간 비동기 호출 시간이다.

실제 순수 커널 시간은 GPU 하드웨어로 측정한 GPU Event 시간(0.045056ms)이다.

2. 첫 호출에 무엇이 포함되었나?

드라이버 가동, 메모리 매핑 같은 GPU의 초기 구동 오버헤드가 포함되어 훨씬 오래 걸렸다.

(단, JIT 컴파일 비용은 이전 실습에서 이미 처리되었으므로 이번 첫 호출에는 포함되지 않았습니다.)

*JIT(Just-In-Time) 컴파일 비용 : 프로그램 실행 중(런타임)에 바이트코드를 기계어로 번역하고 최적화하는 과정에서 소모되는 시간, 메모리, CPU 자원

호출
서로

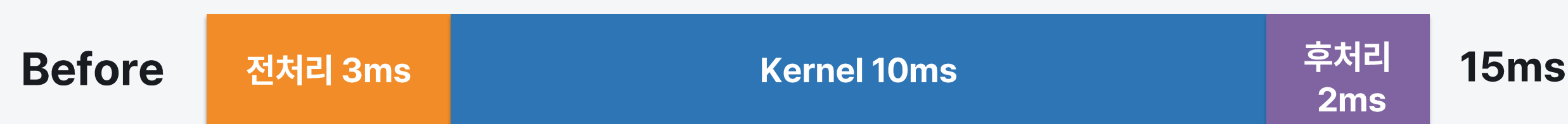
GPU
호출

호출
완료

Kernel이 10배 빨라도 전체 프로그램은 10배가 아닐 수 있다

복사·전처리·후처리가 남으면 End-to-End 가속은 작아진다.

전체 15ms를 단계별로 분해



Kernel Speedup = $10 / 1 = 10$ 배

E2E Speedup = $15 / 6 = 2.5$ 배

비교 설계

- $\text{Speedup}_K = T_{\text{CPU},K} / T_{\text{GPU},K}$
- $\text{Speedup}_{\text{E2E}} = T_{\text{CPU},\text{E2E}} / T_{\text{GPU},\text{E2E}}$
- 분모·분자가 같은 의미의 작업인지 확인
- 포함 단계와 제외 단계를 표의 머리글에 명시
- 제품 판단은 E2E, 커널 튜닝은 Kernel-only

빠른 쪽을 보여 주는 숫자가 아니라 같은 작업을 같은 범위에서 비교하는 숫자가 필요하다.

CPU와 GPU가 같은 입력·dtype·의미를 계산해야 비교가 된다

비교 조건이 하나라도 다르면 속도 차이를 구현 차이라고 말할 수 없다.

공정한 비교의 공통 설계

- ✓ 같은 seed와 입력 데이터
- ✓ 같은 shape·dtype·정밀도
- ✓ 같은 경계 처리와 출력 의미
- ✓ 같은 전처리·후처리 포함 범위
- ✓ 정당한 Compiler 최적화와 실행 조건

Baseline은 “느린 코드”가 아니라 정답과 비교 범위의 기준

성립하는 비교

- CPU Blur 전체 ↔ GPU Blur 전체
- CPU Kernel 구간 ↔ GPU Kernel 구간
- 같은 FP32 허용오차

성립하지 않는 비교

- CPU 전체 ↔ GPU Kernel만
- CPU FP64 ↔ GPU FP16
- 서로 다른 경계 정책

CPU 결과는 정확성 Reference이자 이후 모든 최적화의 비교 기준선이다.

정수는 exact, FP32는 근거 있는 허용오차로 확인한다

정확성을 통과하지 못한 실행시간은 성능 증거가 아니다.

정수·비트 정확 연산

- `gpu == cpu`
- 모든 원소와 경계 값을 exact 비교
- 한 원소라도 다르면 성능 통계에서 제외

부동소수점 연산

- $|gpu - cpu| \leq atol + rtol \cdot |cpu|$
 - ❖ Atol : 절대 허용 오차
 - ❖ Rtol : 상대 허용 오차
- 연산 순서·누적 길이·값 크기 고려
- rtol/atol의 근거와 최대오차 위치 기록

작은 예로 판정

CPU = 0.700000

GPU = 0.700007

오차 = 7×10^{-6}

허용치 = $10^{-6} + 10^{-5} \times 0.7$
= 8×10^{-6}

$7 \times 10^{-6} < 8 \times 10^{-6} \rightarrow \text{PASS}$

허용오차는 결과를 맞게 보이게 하는 숫자가 아니라 연산의 오차 모델이다.

256개씩 나눠 처리하는데, 입력이 257개라면?

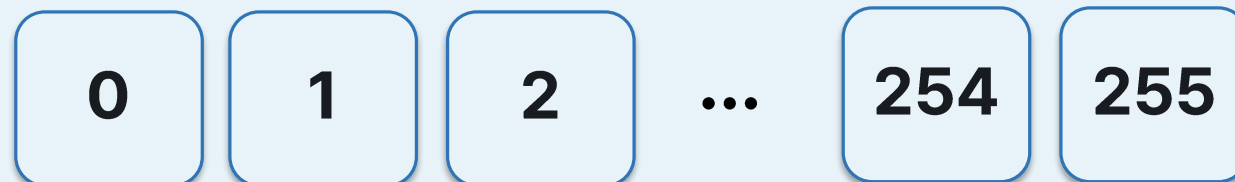
경계 검증 : 마지막에 남은 데이터는 처리하고, 배열 범위 밖에는 접근하지 않는지 확인한다.

입력 257개를 두 블록으로 나누면?

필요한 블록 수 : $\text{cell}(257 / 256) = 2$ (두 개의 블록 실행)

블록 0 (스레드 256개)

데이터 256개를 처리



256개 (모두 유효한 데이터)

블록 1 (스레드 256개)

남은 데이터 1개를 처리



(유효한 데이터)

255개 (배열에 접근하지 않음)

오류 사항

- ❖ **누락 오류** : 두 번째 블록을 실행하지 않으면, 마지막 데이터 1개를 처리하지 못한다.
- ❖ **범위 초과 오류** : 남은 스레드까지 배열에 접근하면 입력 범위 밖을 읽거나 쓸 수 있다.

왜 255, 256, 257 개를 검사할까?

블록당 스레드 256개, 스레드 당 데이터 1개인 경우

입력 크기	만들어지는 상황	확인할 내용
255개	한 블록에서 스레드 1개가 남음	• 남은 스레드가 배열 밖에 접근하지 않는가?
256개	한 블록에 정확히 맞춤	• 마지막 데이터까지 모두 처리하는가?
257개	두 번째 블록이 필요함	• 마지막 데이터 1개를 처리하는가? • 남은 스레드가 배열에 접근하지 않는가?

선택한 블록 크기보다 하나 적게, 정확히 같게, 하나 많게 검사한다.

샘플 몇 개를 눈으로 보는 것이 아니라 설계를 깨뜨리기 쉬운 입력을 체계적으로 생성한다.

[실습 W03-L03] 경계 입력과 정확성

```
boundary_block = 256
boundary_sizes = [0, 1, 255, 256, 257, 513]
def grid_for(n, block):
    return (n + block - 1) // block
expected_grid = [grid_for(n, boundary_block) for n in boundary_sizes]
print(list(zip(boundary_sizes, expected_grid)))
assert grid_for(257, 256) == 2
print("-----")
run_w3_boundary()
```

256에서 맞던 grid 계산이 257에서도 맞을까?

예상:

관측 후 수정:

[실습 W03-L03] 경계 입력과 정확성

```
[(0, 0), (1, 1), (255, 1), (256, 1), (257, 2), (513, 3)]
```

```
-----  
N=0: 커널 호출 없이 빈 결과 PASS
```

```
잘못된 grid: 1 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 255 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 256 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 257 불일치 수: 1 첫 위치: [256]
```

```
수정 후 PASS
```

```
잘못된 grid: 513 불일치 수: 1 첫 위치: [512]
```

```
수정 후 PASS
```

256에서 맞던 grid 계산이 257에서도 맞을까?

맞다. 그리드 계산 공식(grid_for) 자체는 257일 때 2개로 올바르게 계산한다.

다만, CUDA 커널 내부의 잘못된 인덱스 조건 때문에 계산 불일치(에러)가 발생하게 된다.

[실습 W03-L03] 경계 입력과 정확성

```
[(0, 0), (1, 1), (255, 1), (256, 1), (257, 2), (513, 3)]
```

```
-----  
N=0: 커널 호출 없이 빈 결과 PASS
```

```
잘못된 grid: 1 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 255 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 256 불일치 수: 0 첫 위치: []
```

```
수정 후 PASS
```

```
잘못된 grid: 257 불일치 수: 1 첫 위치: [256]
```

```
수정 후 PASS
```

```
잘못된 grid: 513 불일치 수: 1 첫 위치: [512]
```

```
수정 후 PASS
```

잘못된 실행의 마지막 원소가 어떤 값으로 남았나?

연산이 누락되어 초기값(예: 0.0) 그대로 남았다.

257개 중 마지막 1개(인덱스 256번)와 513개 중 마지막 1개(인덱스 512번)가 스레드 경계 문제로 계산되지 못하고 방치되었기 때문이다.

준비·측정·검증·기록을 분리하면 실험을 다시 돌릴 수 있다



각 단계에서 고정해야 하는 것

입력과 seed

Buffer 할당 위치

동기화 범위

반복 횟수

통계 규칙

정확성 합격선

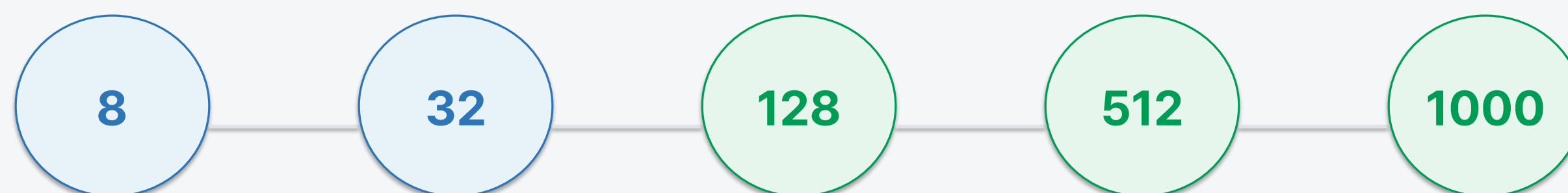
파일 경로

측정 코드와 검증 코드를 섞지 말고 실패한 실행을 통계에서 자동 제외한다.

입력 크기 Sweep 한 번으로 고정비와 Crossover가 보인다

한 크기만 재면 작은 입력과 큰 입력에서 결론이 뒤집히는 것을 놓친다.

권장 Sweep



작은 N: 고정비 지배 ← 교차점 → 큰 N: 병렬 처리 이득

- 각 N마다 CPU/GPU Kernel/E2E를 같은 반복 규칙으로 측정
- 교차점 앞뒤의 결론과 오차 범위를 함께 저장

CSV에 남길 최소 열

N / shape / dtype

cpu_e2e_ms

gpu_kernel_ms

gpu_e2e_ms

median / p95

correct / max_error

GPU·CUDA·build_id

Sweep의 축 단위와 고정 조건을 기록하지 않으면 교차점의 의미도 사라진다.

빠름·맞음·재현 가능함을 한 행에서 확인한다

환경·입력·정확성·측정 범위·통계를 같은 표에 남긴다.

N	CPU E2E	GPU Kernel	GPU E2E	E2E Speedup	median	p95	correct	max_error	GPU/CUDA/build
8	0.16	—	1.516	0.11x	1.51	1.55	PASS	0	Colab 실측 / CUDA / run_id
128	2.56	0.21	1.756	1.46x	1.75	1.82	PASS	7e-6	Colab 실측 / CUDA / run_id
1000	20.0	2.00	3.50	5.71x	3.48	3.60	PASS	7e-6	Colab 실측 / CUDA / run_id

성능 열

- CPU/GPU × Kernel/E2E 범위를 분리

정확성 열

- correct=true와 max_error를 함께 저장

재현성 열

- GPU·CUDA·Compiler·build id 기록

특정 Speedup 숫자보다 동일 설계에서 다시 생성되는 표가 더 강한 성능 증거다.

[실습 W03-L04] 입력 크기 Sweep과 프로젝트 기준

1. 실험 조건: 학생 수를 256명에서 100만 명까지 늘려가며 테스트

```
sweep_sizes = [256, 4096, 65536, 1048576]
```

```
sweep_repeat = 30 # 정확한 측정 유지를 위해 30번 반복
```

2. 무슨 실험인지 쉬운 말로 채워넣기

```
project_input = "학생들의 시험지 답안 데이터" # 내 프로젝트 입력을 한 문장으로 바꾸세요
```

```
project_output = "정답과 비교해 채점 완료된 점수 결과" # 정답을 확인할 출력
```

```
project_unit = "학생 1명의 시험지 (옆 사람과 상관없이 혼자 채점 가능)" # 서로 독립적으로 계산할 단위
```

```
project_metric = "시험지를 다 채점하는 데 걸린 전체 시간" # E2E latency
```

```
project_check = "선생님 혼자 채점하기(CPU) vs 반 전체 학생이 동시에 채점하기(GPU) 비교"
```

3. 출력 후 컴퓨터 성능 대결 시작

```
print(project_input, project_output, project_unit)
```

```
print(project_metric, project_check)
```

```
run_w3_sweep()
```

[실습 W03-L04] 입력 크기 Sweep과 프로젝트 기준

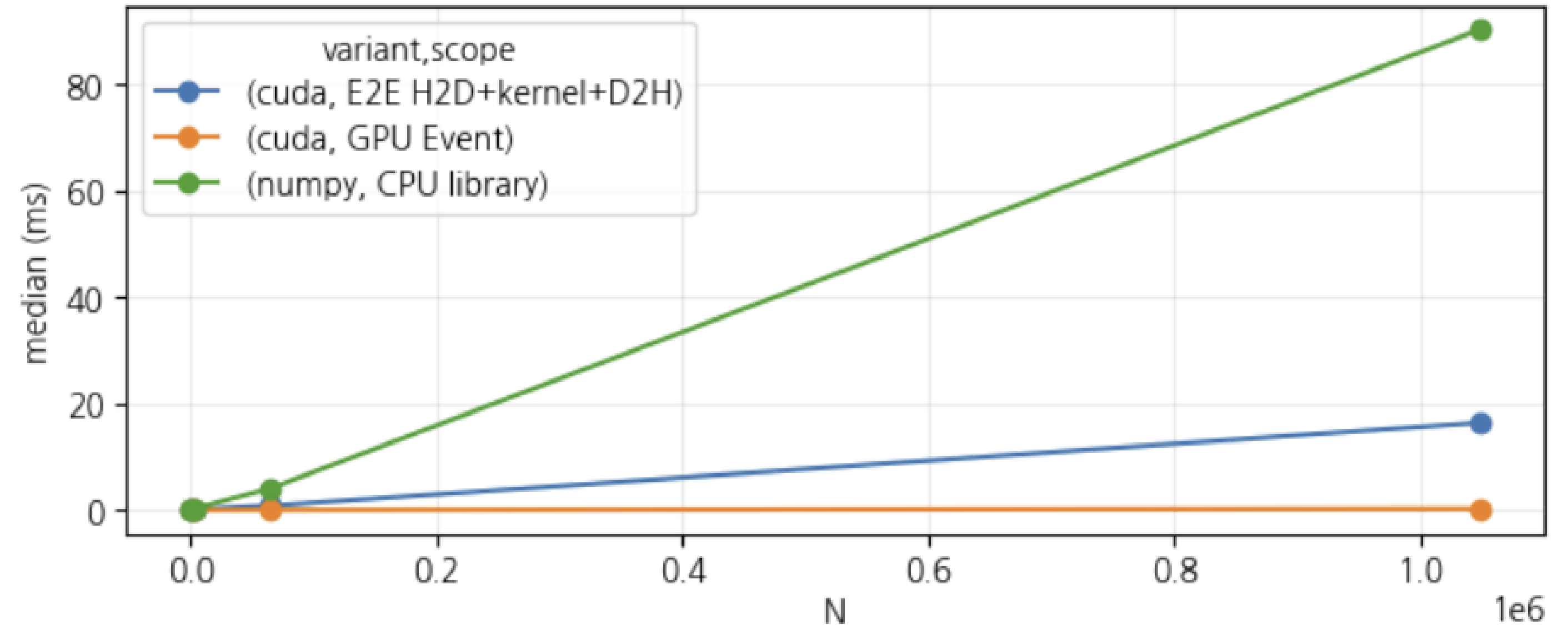
학생 256명 | 1명당 10문항
CPU · GPU 채점 시간을 각각 30회 측정합니다.

학생 4,096명 | 1명당 10문항
CPU · GPU 채점 시간을 각각 30회 측정합니다.

학생 65,536명 | 1명당 10문항
CPU · GPU 채점 시간을 각각 30회 측정합니다.

학생 1,048,576명 | 1명당 10문항
CPU · GPU 채점 시간을 각각 30회 측정합니다.

	variant	scope	n	repeat	median_ms	p95_ms
0	numpy	CPU library	256	30	0.033683	0.069014
1	cuda	GPU Event	256	30	0.020512	0.044363
2	cuda	E2E H2D+kernel+D2H	256	30	0.102503	1.146771
3	numpy	CPU library	4096	30	0.292915	0.318967
4	cuda	GPU Event	4096	30	0.019952	0.033267
5	cuda	E2E H2D+kernel+D2H	4096	30	0.134686	0.157954
6	numpy	CPU library	65536	30	4.082402	8.994079
7	cuda	GPU Event	65536	30	0.028800	0.038621
8	cuda	E2E H2D+kernel+D2H	65536	30	0.781083	1.265729
9	numpy	CPU library	1048576	30	90.337563	176.701186
10	cuda	GPU Event	1048576	30	0.209008	0.227755
11	cuda	E2E H2D+kernel+D2H	1048576	30	16.393639	28.072223



좋은 벤치마크는 네 질문에 즉시 답한다

범위·시간축·통계·정확성이 하나라도 비어 있으면 결과를 해석할 수 없다.

무엇을 잴는가?

Kernel-only / Stage / End-to-End

언제 끝났는가?

Host 동기화 / Event 완료 / Multi-Stream 신호

어떤 분포인가?

Cold·Warm / median / p90·p95

결과가 맞는가?

CPU Reference / 허용오차 / 경계 입력

정답·성능·측정 범위를 섞지 말고, 각각의 합격 조건을 독립적으로 기록한다.

PART 3

실행 가능한 프로젝트 설계를 만드는 법

입력·출력·병렬 단위·전송 경계·예상 병목·정확성·Latency/Throughput을 한 장의 흐름도와 한 표로 고정한다.

프로젝트 설계서는 기술 목록이 아니라 실행 가능한 문제 설계다

입력 설계

shape·dtype·크기 범위·생성/수집 방식

출력 설계

의미·단위·순서·경계 정책

병렬 단위

Thread·Block·Batch가 맞는 독립 작업

의존성·전송

직렬 단계와 H2D/D2H 경계

예상 병목

Memory·Compute·Transfer·Control
+ 근거

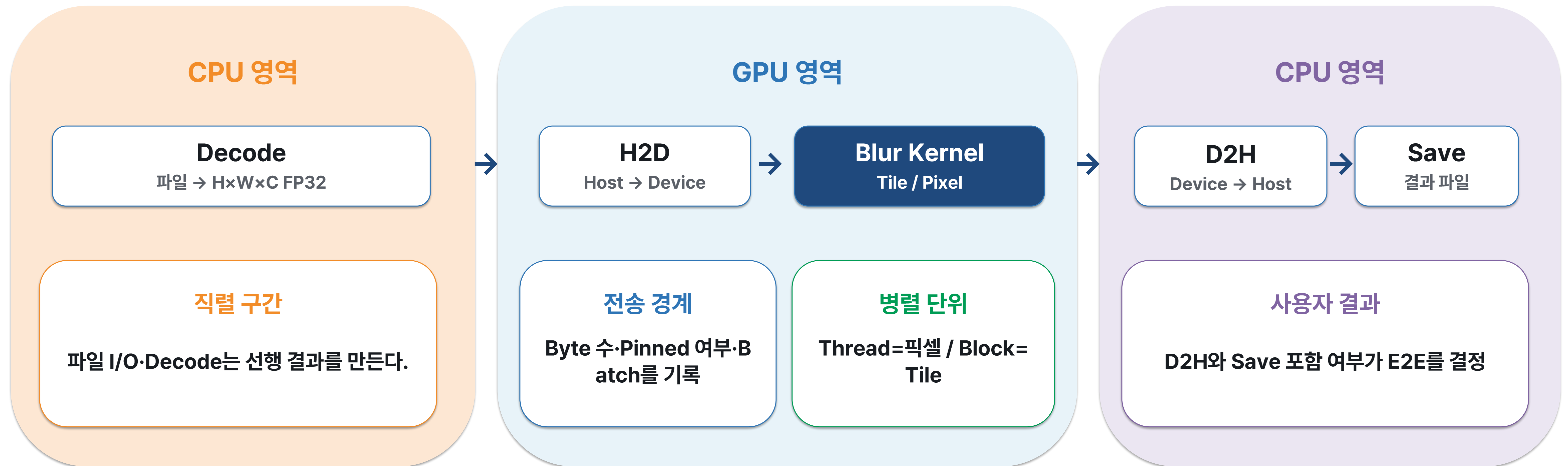
검증·지표

CPU Reference·오차·Latency/Throughput

최소 필드 = I/O + 병렬 단위 + 의존성 + 전송 경계 + 예상 병목 + 검증 + 지표

한 장의 흐름도에 형상·소유자·전송·측정 범위를 적는다

CPU와 GPU의 경계를 그리면 H2D/D2H와 직렬 단계가 보인다.



각 화살표에 데이터 형상·Byte·소유자·동기화·측정 포함 여부를 적는다.

“느릴 것 같다”가 아니라 제한 자원과 근거를 적는다

Memory

- 낮은 연산/Byte
- 연속·비연속 접근
- 대역폭 포화 예상

GB/s·L2 hit·dram throughput

Compute

- 높은 연산/Byte
- 데이터 재사용
- 산술 처리량 제한 예상

FLOP/s·SM utilization

Transfer

- H2D/D2H 비중 큼
- 작은 Batch
- PCIe 고정비 지배

H2D/D2H ms·Batch

Control

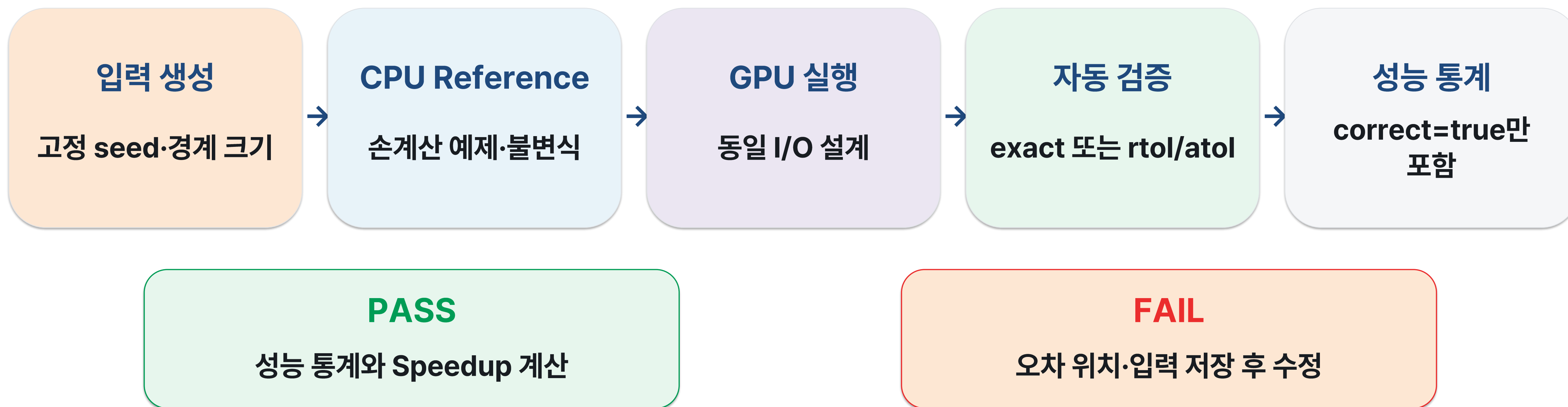
- 불규칙 분기
- 간접 주소
- 활성 Lane 감소

branch efficiency·active lanes

예상 병목은 결론이 아니라 프로파일링과 Sweep으로 반증 가능한 가설이다.

정확성은 성능보다 먼저 통과해야 하는 독립된 합격 조건이다

CPU Reference·경계 입력·dtype별 허용오차가 GPU 결과의 합격선을 정의한다.



최대오차·오차 위치·실패 입력까지 저장해야 다음 실행에서 원인을 재현할 수 있다.

Latency와 Throughput은 서로 다른 목적함수다

한 요청의 대기시간과 단위시간 처리량 중 프로젝트 사용자가 기다리는 값을 먼저 고른다.

Latency · 한 요청의 대기시간

- 요청 시작부터 결과를 받을 때까지의 시간
- median뿐 아니라 p90/p95 Tail latency 확인
- 실시간 미리보기·대화형 서비스에 중요
- 단위: ms/request

Throughput · 단위시간 처리량

- wall time 동안 완료한 작업량
- Batch·Pipeline으로 높일 수 있음
- 야간 100만 장 처리·대규모 ETL에 중요
- 단위: images/s, items/s

Batch 1 → 64: Throughput 3배 ↑ / p95 Latency 10배 ↑ → 서비스 목적에 따라 채택 여부가 달라진다

주지표 1개와 보조지표 2개를 정하고 Batch·동시성 정책을 함께 명시한다.

밝기·Blur·누적합 중 무엇을 먼저 GPU로 옮길까?

독립 단위·연산량·재사용·전송·직렬 의존성·실측 교차점을 함께 본다.

밝기 조정	픽셀 독립 ↑	연산/Byte ↓	재사용 ↓	작은 N은 CPU 유지	2순위
3×3 Blur	픽셀 출력 독립 ↑	연산/Byte 중간	이웃 재사용 ↑	N=1,000에서 E2E 개선	1순위
Prefix Sum	단계 의존성	병렬 단계 필요	동기화 비용	알고리즘 재설계 선행	3순위

선택한 후보뿐 아니라 탈락 후보와 결론을 뒤집는 입력 크기·측정 범위도 기록한다.

좋은 설계서는 입력·독립단위·직렬부·정답·지표를 말한다

3×3 Blur 프로젝트를 실행 가능한 설계로 바꾼다.

프로젝트 설계 예시

입력	1,000장 × 1024×1024 · FP32 · seed=42
출력	3×3 Blur 결과 · reflect 경계 정책
독립 단위	Thread=출력 픽셀 · Block=2D Tile
직렬 구간	파일 Decode와 결과 Save
전송 경계	Batch H2D → Kernel → Batch D2H
예상 병목	초기 Memory/Transfer, Tile 재사용 후 재검증
정답 기준	CPU Blur · rtol=1e-5 · atol=1e-6
주지표	Throughput(images/s), 보조: p95 E2E·max_error

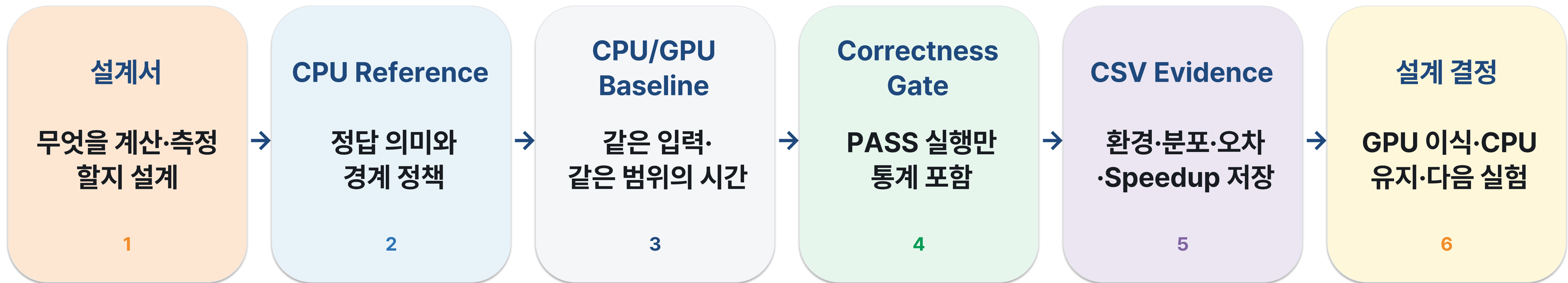
검토자가 확인할 수 있는 것

- ✓ 같은 입력을 다시 만들 수 있음
- ✓ CPU/GPU 경계와 전송 횟수를 알 수 있음
- ✓ 병렬 단위와 직렬 구간을 알 수 있음
- ✓ 정확성 PASS/FAIL을 자동 판정할 수 있음
- ✓ Latency/Throughput 중 성공 기준이 명확함
- ✓ Baseline CSV의 열을 미리 정의할 수 있음

코드 작성 전에 비교 설계가 완성되어야 결과가 무엇을 증명하는지 알 수 있다.

설계서→Reference→Baseline→정확성→CSV→결정이 끊기지 않아야 한다

각 결과가 다음 판단의 입력이 되도록 파일과 필드를 연결한다.



결정 문장: "N≥___, correct=true, p95 E2E≤___ 조건에서 GPU 경로를 사용하고 그 외에는 CPU 경로를 유지한다."

선택 근거와 탈락 근거를 모두 남겨야 다음 최적화가 왜 필요한지 추적할 수 있다.